

UNIVERSIDAD CATÓLICA DE TEMUCO

Informe técnico del analizador léxico para un subconjunto de Prolog

Teoría de la Computación

Dato	Información
Código del curso	INFO1148
Asignatura	Teoría de la Computación
Integrantes	Jose Jimenez Saavedra — jose.jimenez2023@alu.uct.cl Franco Oyarzo Calisto — foyarzo2023@alu.uct.cl Demian Quezada Pasten — dquezada2023@alu.uct.cl
Profesor	Prof. M. Lévano
Periodo académico	Segundo semestre de 2026

Temuco, septiembre de 2026

Resumen

El trabajo presenta el diseño, la implementación y la validación de un analizador léxico para un subconjunto de Prolog. La solución, desarrollada en Python, recorre la fuente de izquierda a derecha y reconoce átomos, variables, números, cadenas, operadores y delimitadores. Los espacios y comentarios se descartan sin perder el seguimiento de línea y columna. Para resolver lexemas con prefijos compartidos se aplica máxima coincidencia; además, los operadores alfabéticos `is` y `mod` se clasifican después de consumir el identificador completo. Cada token conserva su tipo, lexema, posición y, cuando corresponde, un índice en la tabla de lexemas. El signo de los números se trata como operador independiente, de modo que no se confunde el reconocimiento léxico con la interpretación sintáctica. La validación incluye 40 pruebas automatizadas, un programa correcto y otro con errores recuperables. Las pruebas finalizaron sin fallos; el archivo válido produjo 94 tokens y ningún error, mientras que el archivo inválido produjo 30 tokens y seis diagnósticos. El alcance no incluye análisis sintáctico, semántica ni compatibilidad completa con ISO Prolog.

Palabras clave: análisis léxico, Prolog, autómata finito, máxima coincidencia, tabla de lexemas.

1 Introducción

El análisis léxico transforma una secuencia de caracteres en unidades que pueden ser tratadas por las etapas posteriores de un compilador o intérprete. En este trabajo, el problema consiste en recorrer archivos escritos en un subconjunto de Prolog, distinguir las categorías definidas y asociar a cada token su posición inicial. El recorrido también debe descartar comentarios y espacios sin alterar el cálculo de línea y columna, además de informar errores léxicos sin detenerse ante el primer carácter inválido.

La dificultad no se reduce a comparar caracteres con una lista. Varias categorías comparten prefijos: `\==` comienza como `\=`, `=..` comienza como `=` y el átomo `island` contiene el prefijo `is`. Si el analizador aceptara el primer patrón disponible, fragmentaría lexemas que pertenecen a una sola unidad. Por esa razón, la solución conserva la aceptación provisional más larga y aplica una prioridad estable cuando dos reglas pueden reconocer la misma entrada, criterio habitual en la construcción de analizadores léxicos [1].

El informe define primero el subconjunto admitido y sus expresiones regulares. Luego relaciona esas definiciones con AFD para identificadores, números y operadores, e incluye una determinización y una minimización representativas. Las secciones posteriores describen la arquitectura, la tabla de lexemas, el tratamiento de errores y los resultados del corpus automatizado. De este modo, la especificación formal y el comportamiento del programa pueden contrastarse sobre los mismos casos.

2 Objetivos

2.1 Objetivo general

Desarrollar y validar un analizador léxico para un subconjunto definido de Prolog, relacionando expresiones regulares y autómatas finitos con una implementación en Python capaz de generar tokens posicionados y diagnósticos recuperables.

2.2 Objetivos específicos

Los objetivos específicos consisten en definir las categorías admitidas mediante patrones, ejemplos, atributos y reglas de prioridad; modelar categorías representativas con autómatas finitos y documentar una conversión de AFN a AFD junto con su minimización; implementar el recorrido de la fuente, la tabla de lexemas y

los diagnósticos de error; y comprobar el comportamiento del analizador mediante pruebas automatizadas y dos archivos completos de entrada.

3 Alcance del analizador

La especificación restringe los identificadores y dígitos a caracteres ASCII. Esta decisión mantiene explícitas las clases de transición del autómata y acota la implementación al subconjunto pedido, aunque SWI-Prolog admite identificadores Unicode más amplios [4]. Los literales citados sí pueden contener otros caracteres, siempre que no introduzcan un salto de línea ni una secuencia de escape fuera de la convención adoptada.

El analizador produce tokens y errores léxicos, pero no determina si esos tokens forman una cláusula correcta. Tampoco interpreta precedencias, realiza unificación, administra ámbitos ni asigna significado a los literales. La Tabla 1 resume esa frontera para evitar que los resultados se presenten como una implementación completa de Prolog.

Cuadro 1: Decisiones de alcance del analizador.

Aspecto	Incluido	Excluido
Identificadores	Átomos iniciados en minúscula; variables iniciadas en mayúscula o guion bajo; <code>_</code> como anónima.	Unicode fuera de literales citados.
Números	Enteros, decimales con dígitos a ambos lados del punto y exponentes opcionales.	Bases no decimales, racionales, infinidades y NaN.
Texto	Átomos con comillas simples y cadenas dobles; escapes <code>\n</code> , <code>\t</code> , <code>\r</code> , <code>\\</code> , <code>\'</code> y <code>\"</code> .	Salto de línea sin escapar y otros escapes.
Operadores	Cláusulas, consultas, unificación, comparación, aritmética y control señalados en la tarea.	Operadores definidos por el usuario.
Comentarios	<code>%</code> hasta fin de línea y <code>/* ... */</code> .	Comentarios de bloque anidados.
Procesamiento	Tokenización, posiciones, tabla de lexemas y recuperación léxica.	Sintaxis, precedencia, unificación, ámbitos y semántica.

4 Especificación léxica

Se define $L = [a-z]$, $U = [A-Z]$, $D = [0-9]$ y $C = [A-Za-z0-9_]$. Las expresiones de la Tabla 2 usan una notación compatible con `re` de Python. En la implementación, los patrones compilados se aplican desde la posición actual mediante `match`; los literales citados y los comentarios de bloque se recorren carácter por carácter para conservar el fragmento exacto cuando falta un cierre. Esta estrategia reconoce el mismo lenguaje definido por las expresiones, pero permite diagnósticos más precisos [3].

Cuadro 2: Definición formal del subconjunto.

Token	Descripción	Expresión regular conceptual
ATOMO	Identificador iniciado en minúscula.	<code>[a-z][A-Za-z0-9_]*</code>
ATOMO_CITADO	Texto entre comillas simples con escapes admitidos.	<code>'(?:\\[ntr\\'\\"] [^'\\n])*</code>

Token	Descripción	Expresión regular conceptual
VARIABLE	Identificador iniciado en mayúscula o guion bajo, excepto el lexema aislado _.	[A-Z_][A-Za-z0-9_]*
VARIABLE_ ANONIMA	Guion bajo aislado.	_
ENTERO	Secuencia decimal sin signo.	[0-9]+
REAL	Decimal o exponente; el signo solo se admite en el exponente.	(?:[0-9]+\.[0-9]+(?:[eE][+-]?[0-9]+)? [0-9]+[eE][+-]?[0-9]+)
CADENA	Texto entre comillas dobles con escapes admitidos.	\"(?:\\[ntr\\'\"] \\^\\"\\n\\n)*\"
OPERADOR_ CLAUSULA	Operadores de regla y gramática de cláusulas definidas.	--> :-
OPERADOR_ CONSULTA	Inicio de consulta.	\?-
OPERADOR_ COMPARACION	Unificación, identidad, desigualdad y comparación.	\\== =\\.\\. \\ = = < > = < >
OPERADOR_ ARITMETICO	Operadores simbólicos y alfabéticos admitidos.	// ** [+\\-*/] is mod
OPERADOR_ CONTROL	Negación, corte y disyunción.	\\+ ! ;
DELIMITADOR	Agrupación, lista, separación y cierre de cláusula.	[()\\[\\]\\{\\} ,\\.]
COMENTARIO	Línea o bloque descartado por el analizador.	%[^\\n]* /*(?:s:.*?)*/
ESPACIO	Separadores descartados con actualización de posición.	[\\t\\r\\n\\f\\v]+

Los operadores se limitan a los lexemas exigidos por la tarea. Prolog permite declarar operadores y asociarles precedencia y tipo, pero esa información pertenece al análisis sintáctico y queda fuera de este lexer [5]. En consecuencia, `is` y `mod` se tratan como operadores alfabéticos reservados dentro del subconjunto, mientras que cualquier identificador más largo se conserva como átomo.

Cuadro 3: Ejemplos y atributos asociados.

Token	Ejemplos válidos	Ejemplos inválidos	Atributo
ATOMO	padre, persona_1	Padre, 1persona	Índice en átomos
ATOMO_ CITADO	'Juan Perez', ':-'	'sin cierre	Índice en átomos
VARIABLE	X, Persona, _Tmp	x, 2X	Índice en variables
VARIABLE_ ANONIMA	_	_ como anónima	Sin índice
ENTERO	25, 0	12abc, 2e	Índice en literales
REAL	3.14, 2e-3	3.14.15, 1e+	Índice en literales
CADENA	"hola", "a\\n b"	"sin cierre, "x\\q"	Índice en literales

Token	Ejemplos válidos	Ejemplos inválidos	Atributo
OPERADOR	\==, =.., :-, is	:, @	Sin índice
DELIMITADOR	() [] { } , .	@, #	Sin índice

5 Diseño de los autómatas

Los autómatas de las Figuras 1, 2 y 3 representan las categorías con mayor posibilidad de superposición. Los estados con doble circunferencia son aceptadores. Si un estado aceptador mantiene transiciones de salida, el analizador guarda esa aceptación y continúa leyendo; cuando ya no existe una transición válida, retorna el último estado aceptador alcanzado. Ese procedimiento implementa la máxima coincidencia sin confundirla con la prioridad entre categorías.

AFD para átomos no citados y variables

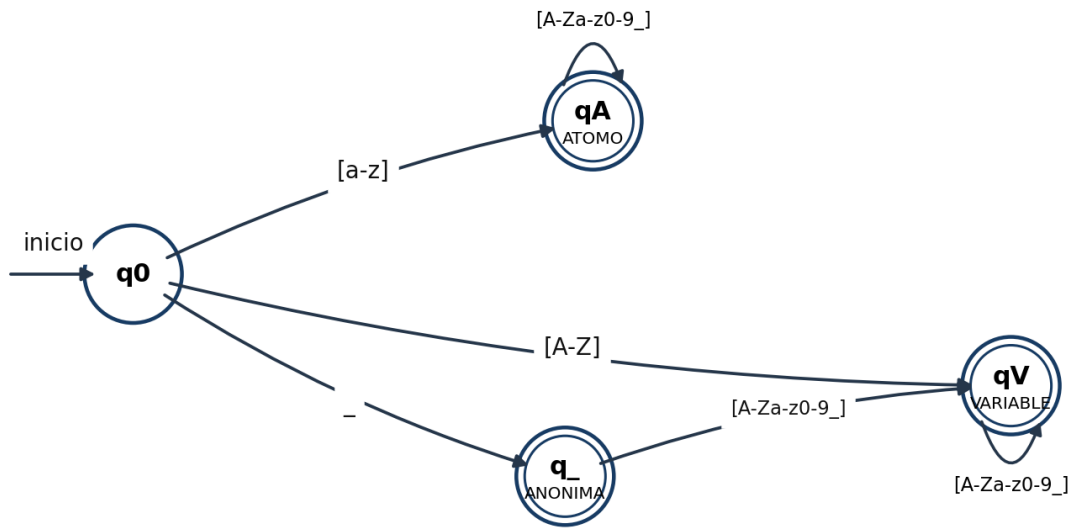
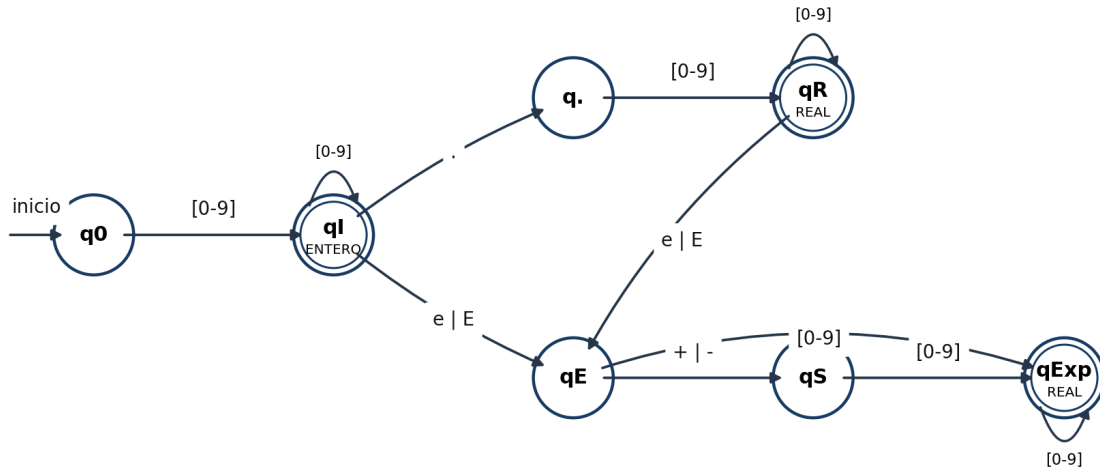
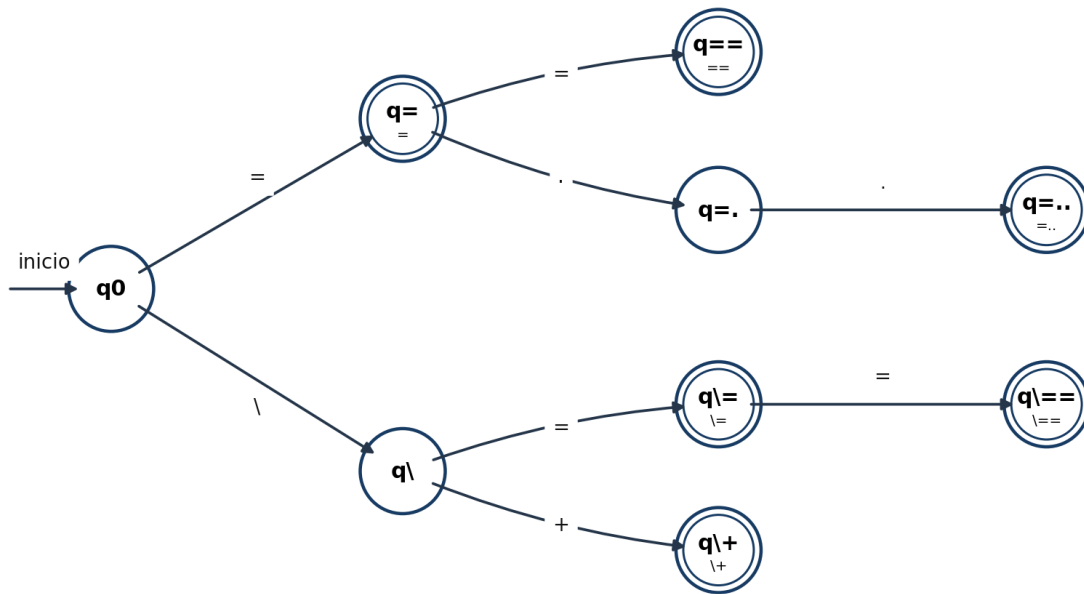


Figura 1: AFD para átomos no citados, variables y variable anónima.

AFD para enteros y reales sin signo**Figura 2:** AFD para enteros y reales sin signo léxico.**Trie de operadores y maxima coincidencia****Figura 3:** Fragmento del trie de operadores con prefijos compartidos.**5.1 Determinización representativa**

Para mostrar la construcción de subconjuntos se utiliza el lenguaje de átomos no citados. Se parte de un AFN deliberadamente no mínimo que divide la primera letra en dos ramas equivalentes. El estado q_0 posee transiciones ε hacia p_0 y r_0 ; desde p_0 , una letra de $[a-m]$ lleva a p_1 , mientras que desde r_0 una letra de $[n-z]$ lleva a r_1 . Los estados p_1 y r_1 aceptan ATOMO y repiten con cualquier carácter de C . La transformación aplica la construcción de subconjuntos y luego refina particiones para obtener estados equivalentes [2].

Cuadro 4: Transiciones del AFN representativo.

Estado	ε	$[a-m]$	$[n-z]$	$U \cup D \cup \{_ \}$
q_0	$\{p_0, r_0\}$	$\emptyset$	$\emptyset$	$\emptyset$
p_0	$\emptyset$	$\{p_1\}$	$\emptyset$	$\emptyset$
r_0	$\emptyset$	$\emptyset$	$\{r_1\}$	$\emptyset$
p_1	$\emptyset$	$\{p_1\}$	$\{p_1\}$	$\{p_1\}$
r_1	$\emptyset$	$\{r_1\}$	$\{r_1\}$	$\{r_1\}$

La clausura- ε inicial es $D_0 = \{q_0, p_0, r_0\}$. Al mover ese conjunto con $[a-m]$ se obtiene $D_1 = \{p_1\}$; con $[n-z]$ se obtiene $D_2 = \{r_1\}$. Cualquier clase sin transición conduce a $D_\perp = \emptyset$. Como p_1 y r_1 repiten con C , los subconjuntos alcanzados se mantienen estables después del primer carácter.

Cuadro 5: Tabla de transición obtenida mediante construcción de subconjuntos.

Estado AFD	$[a-m]$	$[n-z]$	$U \cup D \cup \{_ \}$	Aceptación
$D_0 = \{q_0, p_0, r_0\}$	D_1	D_2	$D_\perp$	No
$D_1 = \{p_1\}$	D_1	D_1	D_1	ATOMO
$D_2 = \{r_1\}$	D_2	D_2	D_2	ATOMO
$D_\perp = \emptyset$	$D_\perp$	$D_\perp$	$D_\perp$	No

5.2 Minimización

La partición inicial es $P_0 = \{\{D_1, D_2\}, \{D_0, D_\perp\}\}$: el primer bloque contiene los estados que aceptan ATOMO y el segundo, los que no aceptan. El bloque no aceptador se divide porque desde D_0 una letra minúscula alcanza aceptación, mientras que desde $D_\perp$ toda entrada permanece en el estado muerto. En cambio, D_1 y D_2 no se separan: ambos producen el mismo token y, para cada clase de entrada de C , permanecen dentro del bloque aceptador. La partición estable es $P_1 = \{\{D_1, D_2\}, \{D_0\}, \{D_\perp\}\}$, por lo que D_1 y D_2 se fusionan en D_A .

Cuadro 6: AFD mínimo para el ejemplo representativo.

Estado mínimo	$[a-m]$	$[n-z]$	$U \cup D \cup \{_ \}$	Aceptación
D_0	D_A	D_A	$D_\perp$	No
$D_A = D_1 \cup D_2$	D_A	D_A	D_A	ATOMO
$D_\perp$	$D_\perp$	$D_\perp$	$D_\perp$	No

El programa no materializa un AFD único en memoria. Utiliza una estrategia equivalente: un despachador selecciona el autómata de la categoría posible a partir del carácter inicial, las expresiones regulares representan los AFD de identificadores y números, y una lista ordenada codifica las hojas del trie de operadores. Los literales citados y comentarios se recorren con estados explícitos de cierre y escape. Todos los reconocedores entregan un lexema aceptado o un diagnóstico, y el ciclo principal conserva una única posición de lectura. Así se mantiene la relación entre el modelo formal y el recorrido implementado sin generar una tabla global de transición.

6 Reglas de prioridad y máxima coincidencia

Cuadro 7: Resolución de superposiciones entre patrones.

Caso	Regla aplicada	Resultado
\== / \= / ==	Los operadores se prueban por longitud descendente. El lexer no confirma \= si existe otro signo igual consecutivo.	\== cuando la entrada es \==.
=.. / =	Tras reconocer = como aceptación provisional, se continúa por dos puntos si están disponibles.	=.. cuando la entrada es =..
:- / :	Solo :- pertenece al subconjunto. Un : aislado genera un diagnóstico.	:- o error en :.
_ / _Variable	Se reconoce primero el identificador completo. Solo el lexema exacto _ se clasifica como anónimo.	VARIABLE_ANONIMA o VARIABLE.
is / island	Se consume la palabra completa y después se consulta la tabla de operadores alfabéticos.	OPERADOR_ ARITMETICO o ATOMO.
-25	El signo queda fuera del patrón numérico y se emite como operador independiente.	- seguido de ENTERO 25.

La implementación aplica estas reglas dentro de un orden de decisión único. Primero reconoce espacios y comentarios, porque no deben producir tokens. Después procesa los literales citados, los números y los identificadores. Los operadores simbólicos se comparan por longitud descendente y los delimitadores se dejan para el final. Si ninguna categoría acepta el carácter actual, se registra un error y se avanza una posición.

Esta secuencia evita dos problemas concretos. Un comentario de bloque debe detectarse antes del operador división, y una palabra completa debe reconocerse antes de decidir si corresponde a **is**, **mod** o a un átomo ordinario. La prioridad, por tanto, no reemplaza la máxima coincidencia; ambas reglas trabajan juntas.

7 Diseño e implementación

7.1 Arquitectura de la solución

Cuadro 8: Componentes de la implementación.

Archivo	Componente	Responsabilidad
src/lexer.py	Token y LexError	Representan salidas inmutables con tipo, lexema, posición y atributo opcional.
src/lexer.py	SymbolTable	Interna átomos, variables y literales; reutiliza el índice de un lexema repetido.
src/lexer.py	Lexer.scan	Recorre la fuente una vez, aplica prioridad, actualiza posiciones y recupera errores.
src/cli.py	Interfaz CLI	Lee UTF-8 y emite salida textual o JSON; retorna 1 si encuentra errores.
tests/test_lexer.py	Pruebas	Reproduce 28 casos válidos y 12 casos inválidos, de posición o recuperación.
corpus/	Entradas completas	Contiene un programa sin errores y otro con varios errores recuperables.

La interfaz carga el archivo como texto UTF-8 y lo entrega a `Lexer.scan`. El recorrido mantiene el índice absoluto, la línea y la columna. Antes de consumir un lexema guarda su posición inicial; después actualiza el contador con el texto realmente leído. Los identificadores y números se reconocen mediante patrones compilados, mientras que los literales citados y comentarios de bloque se procesan manualmente para distinguir cierre, salto de línea y escape. Los operadores simbólicos se consultan en una secuencia ordenada desde el lexema más largo.

El resultado agrupa la lista de tokens, la lista de errores y la tabla de lexemas. La interfaz puede imprimir tokens en el formato solicitado o serializar el resultado completo en JSON. Como cada carácter consumido hace avanzar el índice y la cantidad de operadores es fija, el costo del recorrido crece linealmente con el tamaño de la entrada. Los errores consumen el fragmento problemático y el análisis continúa cuando existe un límite de recuperación identificable.

7.2 Formato de salida

El siguiente fragmento proviene de `corpus/programa_valido.pl`. El quinto campo corresponde al índice de la tabla de lexemas cuando la categoría lo utiliza.

```
<ATOMO, 'padre', 2, 1, 0>
<PARENTESIS_IZQ, '(', 2, 6>
<ATOMO, 'juan', 2, 7, 1>
<COMA, ',', 2, 11>
<ATOMO, 'ana', 2, 13, 2>
<PARENTESIS_DER, ')', 2, 16>
<PUNTO, '.', 2, 17>
```

7.3 Tabla de lexemas

La tabla usa diccionarios que conservan un índice por primera aparición. Los átomos no citados y citados comparten una tabla; las variables poseen otra; y los enteros, reales y cadenas se registran como pares (**tipo**, **lexema**), lo que evita mezclar valores escritos de forma distinta. La variable anónima no se interna porque cada aparición representa una variable nueva y no debe compartir identidad.

Cuadro 9: Ejemplo de internación sin duplicados.

Tabla	Ejemplo	Regla
Átomos	padre → 0; juan → 1; ana → 2	El mismo lexema reutiliza el índice.
Variables	X → 0; Z → 1; Y → 2	_ se excluye.
Literales	(ENTERO, 52) → 0 y (CADENA, "Ana Perez") → 1	El tipo y la escritura forman la clave.

8 Tratamiento de errores léxicos

Cuadro 10: Errores, mensajes y estrategia de recuperación.

Error	Ejemplo	Mensaje	Recuperación
Átomo citado sin cierre	'juan	1:1 Atomo citado sin cierre	Consume hasta salto de línea o EOF y continúa.
Cadena sin cierre	"hola	1:1 Cadena sin cierre	Consume hasta salto de línea o EOF y continúa.
Comentario sin cierre	/* texto	1:1 Comentario de bloque sin cierre	Consume hasta EOF; no existe un límite seguro posterior.
Carácter no admitido	@	1:1 Caracter no admitido	Consume un carácter y continúa.
Número con sufijo	12abc	1:1 Numero mal formado	Consume el fragmento alfanumérico completo.
Exponente incompleto	2e	1:1 Numero mal formado	Consume el fragmento numérico completo.
Puntos repetidos	3.14.15	1:1 Numero mal formado	Consume el fragmento numérico completo.
Escape no admitido	"x\q"	1:1 Secuencia de escape no admitida	Consume hasta la comilla de cierre.

9 Plan de pruebas y resultados

Los resultados fueron verificados el 22 de septiembre de 2026 con Python 3.12. La ejecución automatizada, reproducida en el anexo, completó 40 pruebas con estado OK. De ellas, 28 cubren entradas válidas y 12

comprueban entradas inválidas, posiciones o recuperación. El archivo válido produjo 94 tokens y cero errores; el archivo con errores produjo 30 tokens, seis diagnósticos y código de salida 1.

Cuadro 11: Selección representativa del corpus automatizado.

ID	Entrada	Objetivo	Esperado	Obtenido	Estado
P01	padre	Átomo simple	ATOMO	ATOMO	OK
P03	'Juan Perez'	Átomo citado	ATOMO_ CITADO	ATOMO_ CITADO	OK
P07	_	Anónima sin índice	VARIABLE_ ANONIMA	VARIABLE_ ANONIMA	OK
P10	2e-3	Real con exponente	REAL	REAL	OK
P16	\==	Máxima coincidencia	\==	\==	OK
P24	is island	Límite de palabra	OP / ATOMO	OP / ATOMO	OK
P27	padre(X) repetido	Sin duplicados	Índices 0 y 0	Índices 0 y 0	OK
P29	'juan	Cierre faltante	Error	Error recuperado	OK
P31	/* texto	Comentario abierto	Error	Error hasta EOF	OK
P33	12abc	Número mal formado	Error	Error recuperado	OK
P35	3.14.15	Dos puntos	Error	Error recuperado	OK
P38	a.\n @	Ubicación	2:3	2:3	OK
P39	1e+	Exponente incompleto	Fragmento 1e+	Fragmento 1e+	OK
P40	12abc, ok.	Recuperación numérica	, ok .	, ok .	OK

9.1 Análisis de resultados

La cobertura comprueba cada familia léxica, los límites entre palabras y operadores, la deduplicación de lexemas y la actualización de líneas y columnas. Los casos P16 y P24 muestran que la prioridad no se resuelve solo por el orden de las alternativas: primero se consume el lexema completo y después se clasifica. Los casos P39 y P40 verifican que un exponente incompleto se reporte como un solo fragmento y que el analizador retome el reconocimiento en el delimitador siguiente.

El archivo con errores confirma una recuperación parcial. Después de @, 12abc, una cadena abierta al final de línea, 3.14.15 y un escape inválido, el lexer vuelve a reconocer tokens posteriores. En cambio, un comentario de bloque abierto consume hasta el final del archivo porque no existe una marca léxica segura que indique dónde habría terminado.

La principal limitación está en el alcance: el programa usa identificadores ASCII y no intenta reproducir toda la sintaxis de SWI-Prolog. Tampoco decide si una secuencia de tokens forma una cláusula válida. En consecuencia, los resultados demuestran conformidad con el subconjunto definido en este informe, no compatibilidad total con ISO Prolog.

10 Conclusiones

El analizador cumple el objetivo planteado para el subconjunto definido: reconoce las categorías obligatorias, conserva la posición inicial de cada token, registra átomos, variables y literales sin duplicados innecesarios, y comunica los errores junto con el fragmento que los produce. Esta conclusión se apoya en 40 pruebas

automatizadas finalizadas sin fallos y en dos ejecuciones completas. El programa válido produjo 94 tokens sin diagnósticos; el programa inválido permitió recuperar 30 tokens y reportó seis errores.

La máxima coincidencia resultó decisiva en los operadores con prefijos compartidos. Sin ella, `\==` podría dividirse como `\=` y `=`, mientras que `=.` podría reducirse prematuramente a `=`. La clasificación posterior de identificadores resolvió un problema distinto: `is` se reconoce como operador aritmético, pero `island` permanece como átomo. También se optó por tratar el signo como operador independiente, decisión que mantiene la etapa léxica separada de la interpretación sintáctica de una expresión.

El alcance sigue siendo limitado de forma deliberada. Se admiten identificadores ASCII, un conjunto acotado de escapes y números decimales, pero no todas las extensiones de SWI-Prolog. Un comentario de bloque abierto consume hasta el final del archivo porque no existe un cierre seguro desde el cual reanudar el recorrido. Como trabajo posterior, la solución podría incorporar identificadores Unicode, otras formas numéricas y la generación automática de tablas de transición a partir de una especificación única. Esas ampliaciones tendrían que acompañarse de nuevas pruebas y de la actualización del modelo formal para conservar la coherencia alcanzada.

11 Referencias

Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2.^a ed.). Pearson.
- [2] Hopcroft, J. E., Motwani, R., y Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3.^a ed.). Pearson.
- [3] Python Software Foundation. (2026). *re — Regular expression operations*. Python 3.14.7 documentation. <https://docs.python.org/3/library/re.html>
- [4] SWI-Prolog. (s. f.). *The SWI-Prolog syntax*. Recuperado el 22 de septiembre de 2026 de <https://www.swi-prolog.org/pldoc/man?section=syntax>
- [5] SWI-Prolog. (s. f.). *Operators*. Recuperado el 22 de septiembre de 2026 de <https://www.swi-prolog.org/pldoc/man?section=operators>

12 Anexos

Cuadro 12: Recursos asociados al informe.

Recurso	Ubicación o enlace
Repositorio Git	<i>Enlace que debe incorporarse desde el repositorio definitivo del grupo.</i>
Código principal	<code>src/lexer.py</code> y <code>src/cli.py</code> .
Pruebas	<code>tests/test_lexer.py</code> .
Corpus	<code>corpus/programa_valido.pl</code> y <code>corpus/programa_con_errores.pl</code> .

12.1 Evidencia de ejecución

```
$ python -m unittest discover -s tests -v
Ran 40 tests in 0.001s
OK

$ python -m src.cli corpus/programa_valido.pl --json
94 tokens | 0 errores | codigo de salida 0

$ python -m src.cli corpus/programa_con_errores.pl --json
30 tokens | 6 errores | codigo de salida 1
```

12.2 Registro de contribuciones

El registro definitivo debe extraerse del historial del repositorio, ya que la trazabilidad exige asociar actividades con commits existentes. Los campos se mantienen abiertos porque el material revisado no incluye un repositorio Git ni evidencia que permita atribuir tareas individuales. Cada integrante debe conocer la solución completa, aunque el desarrollo se distribuya.

Cuadro 13: Registro que debe completarse con el historial Git del grupo.

Integrante	Actividad realizada	Commit o evidencia
Jose Jimenez Saavedra	_____	_____
Franco Oyarzo Calisto	_____	_____
Demian Quezada Pasten	_____	_____